Technical Guide

GNN-Based Epidemic Source Detection on Temporal Contact Networks

Master's Thesis Implementation Reference

Author: [Your Name]

March 13, 2026

Abstract

This document is a self-contained technical reference for the master's thesis project on graph neural network (GNN) based epidemic source detection on temporal contact networks. It covers every layer of the software stack: the stochastic SIR epidemic model and its C-based simulation engine, the ve distinct graph representations derived from a temporal contact sequence, the ve GNN architectures (StaticGNN, TemporalGNN, BacktrackingNetwork, DBGNN, and DAG-GNN), the training pipeline with its loss function and early-stopping scheme, the evaluation metrics, the non-ML baselines, and the experimental design including the two parameter sweeps. Mathematical derivations are given in full, code listings are drawn directly from the implementation, and the rationale behind every design decision is explained.

Contents

1 Introduction and Problem Formulation 5
- 1.1 Motivation . . . 5
- 1.2 The Temporal Dimension . . . 5
- 1.3 Formal Problem Statement . . . 6
- 1.4 Pipeline Overview . . . 6
- 1.5 Research Hypotheses . . . 6

2 The SIR Epidemic Model 8
- 2.1 Compartmental Models . . . 8
- 2.2 Continuous-Time SIR on Static Networks . . . 8
- 2.3 Discrete-Time SIR on Temporal Contact Sequences . . . 8
  - 2.3.1 The Temporal Constraint . . . 9
- 2.4 SIR Parameters Used in Experiments . . . 9
- 2.5 Source Filtering: Possible Sources . . . 10

3 Temporal Contact Networks 11
- 3.1 Formal De nition . . . 11
- 3.2 Causal Walks . . . 11
- 3.3 Real-World Networks Used . . . 12

4 Source Detection: Problem De nition and Likelihood 13
- 4.1 Bayesian Formulation . . . 13
- 4.2 Discriminative Surrogate . . . 13
- 4.3 The Susceptible Masking Constraint . . . 13
- 4.4 Log-Likelihood Correction . . . 14

5 Graph Neural Networks: Foundations 15
- 5.1 The Message Passing Framework . . . 15
- 5.2 Speci c Convolution Variants . . . 15
  - 5.2.1 GraphSAGE (SAGEConv) . . . 15
  - 5.2.2 GraphConv . . . 16
  - 5.2.3 Why Output Log-Probabilities? . . . 16

6 Graph Representations for Temporal Networks 17
- 6.1 Representation 1: Static Projection . . . 17
- 6.2 Representation 2: Temporal Activation Patterns . . . 17
- 6.3 Representation 3: Time-Sliced Snapshots . . . 18
- 6.4 Representation 4: De Bruijn Graph . . . 18

6.4.1 Construction . . . 18
6.4.2 Why Sentinel Nodes? . . . 18
6.4.3 Tensor Layout . . . 19
6.5 Representation 5: Temporal Event DAG . . . 19
6.5.1 Construction . . . 19
6.5.2 Tensor Layout . . . 19
6.5.3 Complexity . . . 19

7 StaticGNN: Baseline Architecture 21
7.1 Motivation . . . 21
7.2 Architecture . . . 21
7.2.1 Input Representation . . . 21
7.2.2 Graph Convolution . . . 22
7.2.3 Skip Connection . . . 22
7.2.4 Output . . . 22
7.3 Con guration . . . 22
7.4 Complexity . . . 23

8 TemporalGNN: Time-Slice Approach 24
8.1 Motivation . . . 24
8.2 Architecture . . . 24
8.2.1 Layer Structure . . . 24
8.2.2 Implementation Note . . . 24
8.2.3 Key Design Decision: Shared vs. Separate Weights . . . 25
8.3 Con guration . . . 25

9 BacktrackingNetwork: Temporal Activation Kernels 26
9.1 Paper Reference . . . 26
9.2 Motivation . . . 26
9.3 Architecture . . . 26
9.3.1 Initialisation . . . 26
9.3.2 Layer Update (Equations 3 and 4) . . . 27
9.3.3 Why Sum Aggregation? . . . 27
9.3.4 Output . . . 27
9.4 Complete Forward Pass . . . 27
9.5 Con guration . . . 28
9.6 Memory and Complexity . . . 28

10 DBGNN: De Bruijn Graph Neural Networks 29
10.1 Paper Reference . . . 29
10.2 Core Idea: Higher-Order Causal Paths . . . 29
10.3 Feature Initialisation . . . 29
10.4 DBGNNLayer: Message Passing on the De Bruijn Graph . . . 30
10.4.1 Scatter-Add Implementation . . . 30
10.5 Readout . . . 31
10.6 Con guration . . . 31
10.7 Complexity Discussion . . . 31

11 DAG-GNN: Temporal Event Graph GNN 32

11.1 Paper References 32
11.2 Motivation: Why a DAG? 32
11.3 Feature Initialisation 32
11.4 DAG Reversal and Backward Propagation 33
11.5 DAGConvLayer 33
11.6 Readout: Events to Original Nodes 33
11.7 Handling Isolated Nodes 34
11.8 Con guration 34
11.9 E ect of the t Parameter 34

12 Training Pipeline 35
12.1 Data Structures 35
12.1.1 TSIRData 35
12.1.2 SIRDataset: Constructing Training Samples 35
12.2 Train/Validation Split 36
12.3 Training Loop 37
12.4 Inference on Ground-Truth Simulations 37
12.5 Repetitions 37

13 Evaluation Metrics 39
13.1 Top-k Accuracy 39
13.2 Inverse Rank Score 39
13.3 Normalised Brier Score 39
13.4 Credible Set Coverage 40
13.5 Normalised Entropy 40
13.6 Practical Reporting 40

14 Baselines 41
14.1 Uniform 41
14.2 Degree Centrality 41
14.3 Closeness Centrality 41
14.4 Jordan Center 42
14.5 Betweenness Centrality 42

15 Experimental Setup and Hypotheses 43
15.1 Network and SIR Parameter Combinations 43
15.1.1 Standard Experiments 43
15.1.2 Sweep: Varying Transmission Rate (H2) 43
15.1.3 Sweep: Varying Observation Time (H3) 44
15.2 Wandb Artefact Lineage 44

16 Implementation Details and Code Architecture 45
16.1 Model Registry 45
16.2 Con guration System 45
16.3 Binary Data Storage 46
16.4 Batching Strategies 47
16.5 Device Management 47
16.6 Reproducibility 47
16.7 Directory Structure Summary 48

17 Conclusion 49
17.1 Summary of Technical Content 49
17.2 Model Comparison at a Glance 50
17.3 Open Questions and Future Directions 50

Chapter 1

Introduction and Problem Formulation

1.1 Motivation

Epidemic source detection is the problem of identifying, after an outbreak has partially or fully unfolded, which individual node in a contact network most likely initiated the spreading process. The practical stakes are high: in a hospital ward an unidentified index patient can sustain a nosocomial infection chain; in an organisation an undetected patient zero prolongs quarantine decisions; in digital contagion (misinformation, computer malware) source identification enables targeted countermeasures. Classical network epidemiology provides analytical upper bounds and centrality heuristics, but these are derived under strong structural assumptions (trees, homogeneous mixing, infinite-graph limits) that break down on real finite contact networks.

Graph neural networks offer a data-driven alternative: by training on many simulated outbreaks one can learn a discriminative function that maps an observed SIR snapshot directly to a probability distribution over candidate sources without assuming any particular network structure.

1.2 The Temporal Dimension

Standard GNN-based source detection approaches operate on the static projection of the contact network, aggregating all contacts across time into a single undirected graph. This discards the most informative signal: the ordering of contacts. On a temporal contact network the set of causally possible transmission paths is strictly smaller than the set of paths in the static projection, because a chain $u \to v \to w$ can only transmit an infection if the contact (u, v) precedes the contact (v, w) in real time. An architecture that is unaware of this constraint must learn to ignore spurious paths from data alone, which is both data-inefficient and prone to generalisation failures when the temporal ordering changes.

Three of the five architectures in this project — BacktrackingNetwork, DBGNN, and DAG-GNN — incorporate temporal ordering explicitly, either through edge feature encoding (BacktrackingNetwork), a higher-order De Bruijn graph (DBGNN), or a temporal event directed acyclic graph (DAG-GNN).

1.3 Formal Problem Statement

Let $G = (V; E^t)$ be a temporal contact network where $V = \{1; \ldots; N\}$ is a fixed node set and $E^t = \{(u; v; t)\}$ is a multi-set of timestamped undirected contact events. We assume discrete time $t \in \{0; 1; \ldots; T\}$.

Definition: Source Detection Problem

Given:

- The temporal contact network $G = (V; E^t)$,
- An observed compartmental state $O = (S_{obs}; I_{obs}; R_{obs}) \in \{0; 1\}^{N \times 3}$ recorded at time t_{end}, where $O_{v;k} = 1$ iff node v is in compartment $k \in \{S; I; R\}$,

Find the most probable source node:

$$s^* = \arg\max_{v \in V} P(\text{source} = v \mid O; G). \tag{1.1}$$

Because computing the exact posterior in (1.1) is #P-hard for general networks (requiring marginalisation over all stochastic trajectories), we instead train a discriminative neural network $f_\theta : O \mapsto \log P_\theta(\text{source} = v \mid O)$ using synthetic ground-truth labels from Monte Carlo (MC) SIR simulations.

1.4 Pipeline Overview

The full codebase implements a three-stage pipeline:

1. Simulation (`main_tsir.py`): Run the C-based temporal SIR simulator for every possible source node $v \in V$ with n_{runs} stochastic realisations (ground-truth data) and n_{mc} MC realisations (training data). Results are stored as binary arrays and logged as a versioned wandb artefact.
2. Training and Inference (`main_train.py`): Load the artefact, build the appropriate graph representation, train a GNN model on the MC simulations (cross-entropy loss), then run inference on the ground-truth simulations to produce per-run source probability vectors.
3. Evaluation (`main_eval.py`): Compute rank scores, top-k accuracy, Brier scores, and credible set sizes, both for GNN models and for non-ML baselines.

1.5 Research Hypotheses

The experiments are designed to test three specific hypotheses:

H1 Causal Path Integrity. Models that preserve higher-order causal paths (DBGNN, DAG-GNN) will outperform models that operate on the static projection (StaticGNN), because temporal ordering strictly constrains which transmission paths are causally feasible.

H2 Granularity Convergence. As the transmission probability decreases, outbreaks become sparser and more dependent on the precise sequence of contacts. The

performance gap between temporally-aware models and the static baseline therefore increases with decreasing .

H3 Temporal Signal-to-Noise Resilience. When observations are made early (small fraction of T elapsed), models that exploit temporal structure degrade more gracefully than static models, because they can use partial causal chain information even when the infection has not yet spread widely.

Chapter 2

The SIR Epidemic Model

2.1 Compartmental Models

The SIR model partitions the node population at every time step into three mutually exclusive compartments:

- S (Susceptible): healthy nodes that can acquire the infection.
- I (Infectious): infected nodes that can transmit to susceptible neighbours.
- R (Recovered / Removed): nodes that have recovered and are permanently immune.

On a network the mean- eld ordinary di erential equations are replaced by stochastic individual-level transition rules. On temporal contact sequences the transitions are further conditioned on whether the relevant edge is active at the time of each potential event.

2.2 Continuous-Time SIR on Static Networks

On a static network with adjacency matrix A, the continuous-time SIR process assigns two independent Poisson processes to each pair of nodes:

$$S_v \xrightarrow{A_{uv}} I_v \quad \text{(infection from neighbour u)} \tag{2.1}$$

$$I_v \xrightarrow{} R_v \quad \text{(spontaneous recovery)} \tag{2.2}$$

Each S-I contact edge res at rate ; each I node recovers at rate . The basic reproduction number for a homogeneous random network is:

$$R_0 = \frac{}{} \langle k \rangle; \tag{2.3}$$

where $\langle k \rangle$ is the mean degree. An outbreak is sustained (endemic) when $R_0 > 1$.

2.3 Discrete-Time SIR on Temporal Contact Sequences

The simulation engine intsir/ uses the discrete-time, event-driven formulation of Holme (2020). At each contact event $(u; v; t)$ drawn from the temporal sequence:

1. If $u \in I$ and $v \in S$: node v is infected with probability (Bernoulli trial).
2. If $v \in I$ and $u \in S$: node u is infected with probability (contacts are undirected).
3. After processing all contacts at time t: each I node recovers with probability (a single Bernoulli trial per infectious node per time step).

Key Insight: Why Bernoulli, not Poisson?

In continuous time infection and recovery are Poisson processes. The discrete approximation replaces each in nitesimal time interval dt with a single time step. The per-contact transmission probability is therefore the probability that a transmission event occurs at one speci c contact, not the rate of a Poisson process. The two are equivalent in the limit $dt \to 0$. The discrete formulation is exact for event-driven temporal contact sequences where each contact is an instantaneous event.

2.3.1 The Temporal Constraint

The critical di erence from a static network model is that node v can only be infected by node u at a contact event (u, v, t) if u is currently in the I compartment at time t. This means:

- A contact (u, v, t_1) where $u \in S$ at t_1 is harmless even if u later becomes infected.
- A contact (u, v, t_2) where $u \in I$ at t_2 can transmit, but only if $v \in S$ at t_2 it cannot retroactively infect v for contacts at $t < t_2$.

This temporal constraint is precisely what the causally-aware graph representations (De Bruijn graph, temporal event DAG) encode explicitly.

2.4 SIR Parameters Used in Experiments

Table 2.1: Default SIR simulation parameters for the `france_office` network. Other networks use identical parameters unless noted.

Parameter	Symbol	Value
Transmission probability		0.30
Recovery probability		0.20
Ground-truth runs per source	n_{runs}	5000
Monte Carlo training runs per source	n_{mc}	500
Observation window start	t_{start}	0
Observation window end	t_{end}	274

The -sweep experiment varies $\in \{0.10, 0.20, 0.30, 0.50, 0.70\}$ while keeping $= 0.20$ xed, altering the e ective reproduction number R_0 accordingly.

2.5 Source Filtering: Possible Sources

Not every simulation run produces a useful training sample. Two binary masks are computed:

maximal_outbreak[s, v]: 1 if node v became infected (i.e., is in I ∪ R) in the maximal (deterministic) outbreak starting from source s. A maximal outbreak ignores stochasticity and follows every possible contact edge — it is an upper bound on reachability.

possible[s, r, v]: 1 if node v is a possible source for the observed state in run r. Formally, v is a possible source if the observed infected set $I_r(t) \cup R_r(t)$ is contained in the reachable set from v under the contact sequence.

The lik_possible array encodes a log-likelihood correction:

$$\text{lik_possible}[s, r, v] = \begin{cases} 0 & \text{if possible}[s, r, v] = 1 \\ +1 & \text{otherwise.} \end{cases} \tag{2.4}$$

This term is subtracted from the model's log-probability scores during inference, driving the score of topologically impossible sources to −1.

Chapter 3

Temporal Contact Networks

3.1 Formal De nition

De nition: Temporal Contact Network

A temporal contact network is a tuple $G = (V; E^t)$ where $V = \{0; \ldots; N \quad 1\}$ is a set of nodes and $E^t = \{(u; v; t) : u; v \in V; t \in \mathbb{Z}_0\}$ is a multi-set of timestamped contact events. Each event $(u; v; t)$ records that nodes u and v were in physical (or digital) proximity at discrete time step t.

In the implementation, temporal networks are stored as NetworkX graphs where each edge $(u; v)$ carries a `times` list attribute containing all time steps at which that pair was in contact:

```
# H.edges(data=True) yields:
# (u, v, {'times': [t1, t2, t3, ...]})
for u, v, data in H.edges(data=True):
    contact_times = data['times']  # list of int
```

Listing 3.1: Edge data structure for temporal graphs.

3.2 Causal Walks

De nition: Time-Respecting Path (Causal Walk)

A time-respecting path (also called a causal walk) from node s to node v is a sequence of contact events $(v_0; v_1; t_1); (v_1; v_2; t_2); \ldots; (v_{k\ 1}; v_k; t_k)$ such that $v_0 = s$, $v_k = v$, and $t_1 \quad t_2 \quad \ldots \quad t_k$ (strictly: $t_i < t_{i+1}$ for transmission to be possible).

Causal walks de ne which infection transmission paths are physically possible. The static projection of a temporal network has far more paths than the causal walk set, because the static graph ignores temporal ordering. The ratio of causal walks to static paths grows smaller as the network becomes more temporal (i.e., as the temporal ordering is more constraining). This motivates the design of architectures that operate on the causal walk structure directly.

3.3 Real-World Networks Used

Table 3.1: Networks used in experiments. All are empirical face-to-face contact networks from the SocioPatterns project unless noted otherwise.

Name	Key	N	$\lvert E^t \rvert$	Description
toy_holme	toy	50	500	Synthetic Holme-Kim network, used for fast debugging
karate_static	karate	34	78	Zachary's Karate Club; static network used as a temporal network with all contacts at t = 0
france_o ce	o ce	232	9 600	InVS13 workplace contacts, 274 hourly time steps
lyon_ward	lyon	75	32 400	Hospital ward contacts
malawi	malawi	86	10 000	Rural village contacts, Malawi
students	students	666	185 000	High-school student contacts

Chapter 4

Source Detection: Problem De nition and Likelihood

4.1 Bayesian Formulation

Applying Bayes' theorem to the source detection problem:

$$P(\text{source} = s \mid O) = \frac{P(O \mid \text{source} = s)\, P(\text{source} = s)}{P(O)}. \tag{4.1}$$

Under the uniform prior $P(\text{source} = s) = 1/N$ for all s, the posterior is proportional to the likelihood $P(O \mid \text{source} = s)$. Computing this likelihood exactly requires summing over all stochastic trajectories from sources that are consistent with O, which is #P-hard.

4.2 Discriminative Surrogate

Rather than computing the likelihood, we train a discriminative model:

$$f_{\ } : O \to \log \hat{P}(\text{source} = v \mid O) \in \mathbb{R}^N; \tag{4.2}$$

parameterised by . The model is trained with the cross-entropy (negative log-likelihood) loss:

$$L(\ ) = \frac{1}{|D|} \sum_{(O^{(i)}, s^{(i)}) \in D} \log \hat{P}(\text{source} = s^{(i)} \mid O^{(i)}); \tag{4.3}$$

where D is the set of (observation, true source) pairs from the MC simulations. This is identical to F.nll_loss in PyTorch when the model outputs log-probabilities.

4.3 The Susceptible Masking Constraint

A node v that is observed in state S (susceptible) at t_{end} cannot be the epidemic source, because the source by de nition became infectious at time $t = 0$ and must therefore have left the S compartment. This hard constraint is enforced by:

$$\text{score}(v) \quad 1 \quad \forall v : O_{\ v;S} = 1. \tag{4.4}$$

All five models apply this mask before the final log_softmax, guaranteeing zero probability for susceptible nodes. In code:

```
# x: [B, N, 3], channel 0 = S state
susceptible_mask = x[..., 0].bool()   # True where node is S
scores = scores.masked_fill(susceptible_mask, float('-inf'))
return F.log_softmax(scores, dim=-1)  # [B, N]
```

Listing 4.1: Susceptible masking (identical in all models).

4.4 Log-Likelihood Correction

After inference, the raw model log-probabilities are corrected by the lik_possible term:

$$\tilde{\ell}(s \mid O^{(r)}) = \log \hat{P}(s \mid O^{(r)}) \quad \text{lik_possible}[r, s]. \tag{4.5}$$

Since lik_possible is $+1$ for topologically impossible sources and 0 for possible ones, this correction drives impossible sources to -1 and leaves possible sources unchanged. It implements an approximation to the prior $P(\text{source} = s)$ based on network reachability.

Chapter 5

Graph Neural Networks: Foundations

5.1 The Message Passing Framework

All GNN architectures in this project are instances of the message passing neural network (MPNN) framework of Gilmer et al. (2017). In the MPNN framework, each layer updates node embeddings $h_v^{(\ell)}$ according to:

$$h_v^{(\ell+1)} = \mathrm{UPDATE}^{(\ell)}\left(h_v^{(\ell)}, \bigoplus_{u \in N(v)} \mathrm{MSG}^{(\ell)}\left(h_v^{(\ell)}, h_u^{(\ell)}, e_{uv}\right)\right), \tag{5.1}$$

where:

- $N(v)$ is the set of neighbours of node v,
- e_{uv} is an optional edge feature vector,
- $\bigoplus$ is a permutation-invariant aggregation operator (sum, mean, or max),
- MSG and UPDATE are learnable functions (typically MLPs or linear layers).

After L layers, the embedding $h_v^{(L)}$ aggregates information from the L-hop neighbourhood of v.

5.2 Specific Convolution Variants

5.2.1 GraphSAGE (SAGEConv)

The mean-aggregator variant of GraphSAGE (Hamilton et al., 2017) is:

$$h_v^{(\ell+1)} = W_1^{(\ell)} h_v^{(\ell)} + W_2^{(\ell)} \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(\ell)}. \tag{5.2}$$

SAGEConv is used in StaticGNN (optionally), TemporalGNN, DBGNN, and DAG-GNN.

5.2.2 GraphConv

The GraphConv layer (Morris et al., 2019), as implemented in PyTorch Geometric:

$$h_v^{(\ell+1)} = W_1^{(\ell)} h_v^{(\ell)} + W_2^{(\ell)} \sum_{u \in N(v)} h_u^{(\ell)} : \tag{5.3}$$

This differs from SAGEConv only in the absence of mean-normalisation on the neighbour sum, making it sensitive to node degree. StaticGNN defaults to GraphConv with sum aggregation (aggr: sum).

5.2.3 Why Output Log-Probabilities?

All models output $\log \hat{P}(s \mid O) \in \mathbb{R}^N$ (log-softmax). This has two advantages:

1. The cross-entropy training loss (F.nll_loss) expects log-probabilities and computes $\log \hat{P}$ directly without an additional log call, improving numerical stability.
2. Multiplying probabilities over independent runs (unlikely to be needed here, but standard practice) is equivalent to adding their logarithms, avoiding floating-point underflow.

Chapter 6

Graph Representations for Temporal Networks

Each GNN architecture requires a speci c graph representation of the temporal network $G = (V; E^t)$. All ve representations are built once per network (not per sample) by functions in gnn/graph_builder.py and stored in a dict of pre-computed tensors.

6.1 Representation 1: Static Projection

The static projection collapses all temporal edges into a single undirected graph:

$$E_{static} = f(u; v) : 9\, t \text{ s.t. } (u; v; t) \; 2 \; E^t g: \tag{6.1}$$

The optional edge weight w_{uv} counts the number of temporal contacts between u and v, normalised to $[0; 1]$:

$$w_{uv} = \frac{jft : (u; v; t) \; 2 \; E^t gj}{\max_{(u^0;v^0)2E_{static}} jft : (u^0; v^0; t) \; 2 \; E^t gj}: \tag{6.2}$$

The output dict has keys edge_index $2 \; Z^{2 \; 2jEj}$ (both directions) and optionally edge_weight $2 \; R^{2jEj}$.

Loss of information: The static projection loses all information about when each contact occurred. Two networks with identical static projections but di erent contact orderings are indistinguishable to a GNN operating on this representation.

6.2 Representation 2: Temporal Activation Patterns

For the BacktrackingNetwork, each edge $(u; v)$ in the static projection is additionally annotated with a binary activation vector:

$$a_{uv} \; 2 \; f0; 1g^T; \quad (a_{uv})_t = 1[(u; v; t) \; 2 \; E^t]; \tag{6.3}$$

where $T = t_{max} + 1$ and $t_{max} = \max_{(u;v;t)2E^t} t$.

This gives the model access to which time steps each edge was active, but not directly to the ordering relative to other edges. The edge attribute matrix is $A \in \{0,1\}^{2|E| \times T}$.

For the france_office network with $T = 275$, each edge carries a 275-dimensional binary vector.

6.3 Representation 3: Time-Sliced Snapshots

The temporal contact sequence is partitioned into non-overlapping time windows of width Δt (group_by_time):

$$\text{snapshot } s = \{(u,v) : (u,v,t) \in E^t,\ \lfloor t/\Delta t \rfloor = s\}. \tag{6.4}$$

The output is a dict edge_indeces[s] mapping each snapshot index s to its edge_index tensor. The TemporalGNN iterates over these snapshots in reverse temporal order (from $s = S-1$ down to $s = 0$), applying a separate SAGEConv weight matrix per snapshot.

6.4 Representation 4: De Bruijn Graph

The De Bruijn graph B encodes higher-order causal paths in the temporal network. It was introduced for temporal network analysis by Qarkaxhija et al. (2022).

6.4.1 Construction

Definition: De Bruijn Graph of a Temporal Network

Given temporal contact sequence E^t, the De Bruijn graph $B = (V_B, E_B)$ is defined as:

- Event nodes: For each contact $(u,v,t) \in E^t$, add a node (u,v,t) to V_B.
- Sentinel nodes: For each original node $n \in V$ and each time $t \in \{t_{min}, t_{max}\}$, add a sentinel node (n,n,t).
- Causal edges: Add directed edge $(x,y,t_1) \to (y,z,t_2)$ whenever $t_2 > t_1$ (the walk $x \to y \to z$ is time-respecting).
- Sentinel connections: Sentinel node (n,n,t) connects to all event nodes that start or end at node n.

The De Bruijn graph is a line graph of the temporal network: each node in B corresponds to a temporal edge (contact) in G, and each edge in B corresponds to a time-respecting path of length 2 in G.

6.4.2 Why Sentinel Nodes?

Sentinel nodes (n,n,t) serve as readout anchors. After message passing on B, the embedding of the end-time sentinel (n,n,t_{max}) aggregates all causal walk information that terminates at node n by time t_{max}. This provides a natural per-node readout without requiring a separate pooling step.

6.4.3 Tensor Layout

The output dict contains:

- db_node_to_original $\in \mathbb{Z}^{|V_B| \times 2}$: maps De Bruijn node i to its original node pair (u_{orig}, v_{orig}). For event node (u, v, t): entry is (u, v). For sentinel (n, n, t): entry is (n, n).
- sentinel_end_indices $\in \mathbb{Z}^{N}$: sentinel_end_indices[n] is the integer index of De Bruijn node (n, n, t_{max}) in V_B.
- is_sentinel $\in \{0, 1\}^{|V_B|}$: boolean mask; True for sentinel nodes (where $u_{orig} = v_{orig}$).
- edge_index $\in \mathbb{Z}^{2 \times |E_B|}$: directed De Bruijn edges.

6.5 Representation 5: Temporal Event DAG

The temporal event graph (TEG) of Saramäki et al. (2019) maps each directed contact event to a node and connects causally compatible events by directed edges.

6.5.1 Construction

Definition: Temporal Event Graph (TEG)

Given E^t (treating undirected edges as two directed events), the TEG $G = (V_T, E_T)$ is:

- Event nodes: For each directed event $(u \to v, t)$ (both directions for undirected contacts), add a node e_{uvt} to V_T.
- Causal edges: Add directed edge $e_{uvt_1} \to e_{vwt_2}$ if $t_2 > t_1$ (and $t_2 - t_1 \le \Delta t$ if a maximum gap Δt is specified). The events share intermediate node v, so the infection could travel $u \to v$ at t_1 and then $v \to w$ at t_2.

Because $t_2 > t_1$ strictly, the TEG has no directed cycles: it is a DAG. This is the property that DAG-GNN exploits for backward propagation.

6.5.2 Tensor Layout

- dag_edge_index $\in \mathbb{Z}^{2 \times |E_T|}$: forward causal edges $e_1 \to e_2$ with $t(e_1) < t(e_2)$.
- event_src_node $\in \mathbb{Z}^{|V_T|}$: departing node u for each event e_{uvt}.
- event_to_node $\in \mathbb{Z}^{|V_T|}$: arriving node v for each event e_{uvt}.
- event_times $\in \mathbb{Z}^{|V_T|}$: time t of each event.

6.5.3 Complexity

The number of event nodes is $|V_T| = 2|E^t|$ (two directed events per undirected contact). The number of DAG edges can be $\Theta(|V_T|^2)$ in the worst case (all events at the same node), but is typically much smaller for real contact sequences. The Δt parameter limits the causal horizon and reduces edge density.

Table 6.1: Summary of the ve graph representations and their properties.

Representation	Used by	Node count	Temporal info	Edge features
Static projection	StaticGNN	N	No	Optional weights
Activation patterns	BacktrackingNetwork	N	Partial	a_{uv} 2 f0; 1g^T
Snapshots	TemporalGNN	N	Ordering	None
De Bruijn graph	DBGNN	2jE ^{t}j + N	Full (causal walks)	None
Temporal event DAG	DAG-GNN	2jE^{t}j	Full (DAG)	None

Chapter 7

StaticGNN: Baseline Architecture

7.1 Motivation

StaticGNN provides the static-projection baseline: it is the strongest possible GNN that ignores temporal ordering. All temporal information (contact times) is discarded; the model sees only the aggregated static graph plus the SIR state snapshot O. If temporally-aware models do not outperform StaticGNN, it suggests that temporal ordering is not needed for source detection on the tested networks.

7.2 Architecture

The StaticGNN is a con gurable module with four stages (implemented in gnn/static_gnn.py):

1. Preprocessing MLP: L_{pre} layers mapping node features $x_v \in \mathbb{R}^3$ to embeddings $h_v^{(0)} \in \mathbb{R}^{D_{pre}}$.
2. Graph Convolutions: L_{conv} GraphConv layers, each followed by optional batch normalization, PReLU activation, and dropout.
3. Postprocessing MLP: L_{post} fully-connected layers at dimension D_{hidden} (identity in dimension).
4. Final Linear Layer: Projects each node embedding to a scalar score, then reshapes to $[B; N]$ and applies log-softmax.

7.2.1 Input Representation

The input to StaticGNN is a batch of SIR snapshots $X \in \mathbb{R}^{B \times N \times 3}$. Each node v in batch item b has features:

$$x_{b;v} = [O_{b;v;S}; O_{b;v;I}; O_{b;v;R}]^{\top} \in \{0; 1\}^3: \tag{7.1}$$

This is a one-hot encoding of the SIR state.

For PyTorch Geometric (PyG) style batching, the $[B; N; 3]$ tensor is attened to $[B \cdot N; 3]$ with a batch vector $b \in \{0; \ldots; B-1\}^{B \cdot N}$ and the edge index is replicated B times with appropriate o sets.

7.2.2 Graph Convolution

The default aggregation in the experiments is aggr: sum (GraphConv):

$$h_v^{(\ell+1)} = \mathrm{PReLU}\left(\mathrm{BN}\left(W_1^{(\ell)} h_v^{(\ell)} + W_2^{(\ell)} \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)}\right)\right): \tag{7.2}$$

With batch normalization and PReLU, a dropout mask is applied after each layer during training.

7.2.3 Skip Connection

When skip: true is set, the raw input features x_v are concatenated to the post-processed node embedding before the final linear layer:

$$\mathrm{score}(v) = W_{\mathrm{out}} \left[h_v^{(L)} ; x_v\right]: \tag{7.3}$$

This allows the final layer to directly combine topological evidence (from $h_v^{(L)}$) with the node's own observed state (from x_v). In practice this consistently improves performance because the susceptible masking constraint means the node's own state is highly informative.

7.2.4 Output

After applying the final layer to each node, the tensor is reshaped from $[B \cdot N, 1]$ to $[B, N]$, the susceptible mask is applied, and log-softmax is computed row-wise:

$$\hat{\ell}_{b,v} = \log \hat{P}(\mathrm{source} = v \mid O_b): \tag{7.4}$$

7.3 Configuration

```yaml
model: static_gnn

train:
  n_mc: 500
  reps: 3
  test_size: 0.30
  batch_size: 128
  epochs: 500
  patience: 10
  lr: 0.001
  weight_decay: 0.0005
  seed: 42

static_gnn:
  num_preprocess_layers: 0
  embed_dim_preprocess: 16
  num_postprocess_layers: 0
  num_conv_layers: 4
  aggr: sum
  hidden_channels: 64
```

```yaml
  dropout_rate: 0.2
  batch_norm: true
  skip: true
  use_edge_weights: false
```

Listing 7.1: StaticGNN con g for france_office (from exp/france_office/static_gnn.yml).

7.4 Complexity

- Parameters: $O(D^2)$ per convolution layer, where $D = 64$ (hidden channels).
- Forward pass time: $O(B \cdot (N \cdot D^2 + |E| \cdot D))$ per layer.
- Memory: $O(B \cdot N \cdot D)$ for node embeddings; $O(|E|)$ for the static edge index (shared across all batch items).

Chapter 8

TemporalGNN: Time-Slice Approach

8.1 Motivation

TemporalGNN (implemented in gnn/temporal_gnn.py) introduces temporal information through snapshot processing: it applies a separate SAGEConv layer to each time-slice of the contact sequence. The key idea is that the network topology at each time step is di erent, so information should propagate along the graph structure that was actually present at that time.

8.2 Architecture

$$h_v^{(t)} = \text{SAGEConv}_t\ h_v^{(t\ 1)}\ ;\ f(u;v)\ 2\ \text{snapshot}\ tg\ ;\quad t = S\ \ 1; S\ \ 2; : : : ; 0; \qquad (8.1)$$

where S is the total number of snapshots and the convolutions are applied in reverse temporal order. Reverse ordering means that the model rst processes the contacts that occurred latest (close to t_{end} when the observation is made) and then propagates backward through earlier contacts. This matches the direction of source detection: from the observed state backward toward the probable source.

8.2.1 Layer Structure

1. Pre-linear: $h_v^{(1)} = \text{ReLU}(W_{pre} x_v)$, projecting from 3 to hidden_channels.
2. Snapshot convolutions: For each snapshot t in reverse order, apply SAGEConv_t (each has its own weight matrices $W_1^{(t)}; W_2^{(t)}$).
3. Post-linear: $\text{score}(v) = W_{post} h_v^{(0)}$, projecting to a scalar. Output shape $[B; N; 1]$ is squeezed to $[B; N]$.
4. Log-softmax over nodes.

8.2.2 Implementation Note

The model has one SAGEConv layer per snapshot, so the number of parameters scales with the number of snapshots. For the france_office network with 274 time steps

and group_by_time=1, this means 274 separate SAGEConv layers. In practice, the group_by_time parameter is used to reduce this to a manageable number of snapshots.

```
def forward(self, x, edge_indeces, edge_attr=None):
    x = F.relu(self.lin_pre(x))
    count = 0
    for t in reversed(edge_indeces.keys()):
        x = F.relu(self.convs[count](x, edge_indeces[t], edge_attr)
            )
        count += 1
    x = self.lin_post(x)
    return F.log_softmax(x, dim=1)
```

Listing 8.1: TemporalGNN forward pass (from gnn/temporal_gnn.py).

8.2.3 Key Design Decision: Shared vs. Separate Weights

TemporalGNN uses separate weight matrices per snapshot (ModuleList of SAGEConv layers). An alternative would be to share weights across snapshots (a single SAGEConv applied at each step). The separate-weight design allows the model to learn snapshot-speci c aggregation patterns for example, that contacts early in the epidemic are more informative about the source than late contacts.

8.3 Con guration

```
model: temporal_gnn

temporal_gnn:
  hidden_channels: 64
  group_by_time: 1
```

Listing 8.2: TemporalGNN con g (exp/france_office/temporal_gnn.yml).

Chapter 9

BacktrackingNetwork: Temporal Activation Kernels

9.1 Paper Reference

Ru et al., Inferring Patient Zero on Temporal Networks via Graph Neural Networks, AAAI 2023. The implementation in gnn/backtracking_network.py follows Figure 2 and Equations 3 6 of the paper.

9.2 Motivation

The BacktrackingNetwork (BN) enriches the static-graph message passing with edge textures: binary vectors $a_{uv} \in \{0,1\}^T$ encoding when each edge was active. The intuition is that the temporal activation pattern of an edge is informative about whether transmission along that edge is plausible given the observed SIR state: if edge (u,v) was only active at $t < t_{\text{infection of } u}$, it cannot have transmitted the infection from u to v.

9.3 Architecture

9.3.1 Initialisation

$$h_v^{(0)} = \phi^v(x_v) = \mathrm{ReLU}(W_v x_v); \quad W_v \in \mathbb{R}^{D \times 3}; \tag{9.1}$$

$$g_{(u,v)}^{(0)} = \phi^e(a_{uv}) = \mathrm{ReLU}(W_e a_{uv}); \quad W_e \in \mathbb{R}^{D \times T}: \tag{9.2}$$

Node features $x_v \in \{0,1\}^3$ are projected to a D-dimensional hidden space. Edge activation patterns $a_{uv} \in \{0,1\}^T$ are projected to the same hidden dimension.

Note that the edge projections $g_{(u,v)}^{(0)}$ are shared across the batch at initialisation, because the activation patterns are network properties (not sample-specific). They become sample-specific at layer 1 when the node features $h_v^{(0)}$ (which are sample-specific) enter the edge update.

9.3.2 Layer Update (Equations 3 and 4)

For layer $\ell = 0, \ldots, L-1$:

Edge update (Eq. 3 of the paper):

$$g^{(\ell+1)}_{(u,v)} = f^{e}\left(g^{(\ell)}_{(u,v)}, h^{(\ell)}_{u}\right), \tag{9.3}$$

where $f^{e} : \mathbb{R}^{2D} \to \mathbb{R}^{D}$ is a fully-connected layer with ReLU. The edge embedding incorporates the current embedding of its source node.

Node update (Eq. 4 of the paper):

$$h^{(\ell+1)}_{v} = \mathrm{ReLU}\left(f^{v}\left(h^{(\ell)}_{v} + \sum_{u \in N(v)} g^{(\ell+1)}_{(u,v)}\right)\right), \tag{9.4}$$

where $f^{v} : \mathbb{R}^{D} \to \mathbb{R}^{D}$ is a separate fully-connected layer with ReLU. The node embedding accumulates incoming edge messages via sum aggregation.

9.3.3 Why Sum Aggregation?

The paper uses sum (not mean) aggregation. This makes the node embedding sensitive to the degree of the node: a node with many incoming edges has a larger aggregation term. For source detection this is appropriate because the source typically has more infected neighbours (higher effective out-degree in the infection tree).

9.3.4 Output

$$\mathrm{score}(v) = W_{\mathrm{final}}\, h^{(L)}_{v} \in \mathbb{R}, \tag{9.5}$$

then susceptible masking and log-softmax produce $\hat{s} \in \mathbb{R}^{B \times N}$.

9.4 Complete Forward Pass

```python
def forward(self, x, edge_index, edge_attr):
    # x: [B, N, 3], edge_attr: [E, T]
    B, N, _ = x.shape

    # 1. Initial projections
    h = self.p_v(x)                                    # [B, N, D]
    g = self.p_e(edge_attr)                            # [E, D]
    g = g.unsqueeze(0).expand(B, -1, -1).contiguous()  # [B, E, D]

    # 2. L layers of BN convolution
    for conv in self.convs:
        h, g = conv(h, g, edge_index)

    # 3. Final projection to scalar score per node
    scores = self.final(h).squeeze(-1)                 # [B, N]

```

```
    # 4. Expert knowledge: susceptible nodes -> -inf
    susceptible_mask = x[..., 0].bool()
    scores = scores.masked_fill(susceptible_mask, float('-inf'))
    return F.log_softmax(scores, dim=-1)            # [B, N]
```

Listing 9.1: BacktrackingNetwork forward pass (from gnn/backtracking_network.py).

9.5 Con guration

```
model: backtracking

train:
  n_mc: 500
  reps: 3
  test_size: 0.20
  batch_size: 128
  epochs: 500
  patience: 30
  lr: 0.001
  weight_decay: 0.0001
  seed: 42

backtracking:
  hidden_dim: 32
  num_layers: 6
```

Listing 9.2: BacktrackingNetwork con g (exp/france_office/backtracking.yml).

9.6 Memory and Complexity

- Edge projection: $W_e \in \mathbb{R}^{D \times T}$ has $D \cdot T$ parameters. For $D = 32$, $T = 275$: approximately 8 800 parameters just for the edge projection.
- Per-layer: Each BNConvLayer has $f^e \in \mathbb{R}^{2D \times D}$ and $f^v \in \mathbb{R}^{D \times D}$, so $3D^2$ parameters. With $L = 6$ layers: $18D^2$.
- Memory: The edge hidden tensor $g \in \mathbb{R}^{B \times E \times D}$ grows linearly in batch size B and edge count E. For large temporal networks (many edges), this can dominate memory usage.

Chapter 10

DBGNN: De Bruijn Graph Neural Networks

10.1 Paper Reference

Qarkaxhija, Perri, and Scholtes, De Bruijn Goes Neural: Causality-Aware Graph Neural Networks for Time Series Data on Dynamic Graphs, arXiv:2209.08311 (2022). Our implementation in gnn/dbgnn.py adapts this architecture for epidemic source detection.

10.2 Core Idea: Higher-Order Causal Paths

The central observation motivating DBGNN is that standard GNNs on the static projection aggregate information from the topological k-hop neighbourhood, whereas for temporal networks the relevant neighbourhood is the causal k-hop neighbourhood the set of nodes reachable by time-respecting paths of length at most k.

By operating on the De Bruijn graph, whose nodes represent individual contact events (temporal edges), DBGNN performs message passing along causal walks of length 2. A node $(x; y; t_1)$ in the De Bruijn graph can send messages to $(y; z; t_2)$ if and only if $t_2 > t_1$, enforcing the causal constraint directly in the graph structure.

10.3 Feature Initialisation

Each De Bruijn node i with original node pair $(u_{orig}; v_{orig})$ is initialised differently depending on whether it is a sentinel or an event node:

Sentinel nodes (where $u_{orig} = v_{orig} = n$):

$$h_i^{(0)} = \mathrm{ReLU}(W_{sent}\, x_n); \quad W_{sent} \in \mathbb{R}^{D \times 3}: \tag{10.1}$$

The projection uses only the SIR state of the single associated original node.

Event nodes (where $u_{orig} \neq v_{orig}$):

$$h_i^{(0)} = \mathrm{ReLU}\left(W_{evt}\, [x_{u_{orig}}; x_{v_{orig}}]^{\top}\right); \quad W_{evt} \in \mathbb{R}^{D \times 6}: \tag{10.2}$$

The projection concatenates the SIR states of both endpoints of the contact event, giving the model access to the states of both nodes that participated in the contact.

In code, both projections are computed for all nodes, then blended using the boolean mask is_sentinel:

```
x_u = x[:, u_orig, :]                    # [B, db_N, 3]
x_v = x[:, v_orig, :]                    # [B, db_N, 3]

h_sent  = F.relu(self.proj_sentinel(x_u))            # [B, db_N, D]
h_event = F.relu(self.proj_event(
    torch.cat([x_u, x_v], dim=-1)))                   # [B, db_N, D]

mask = is_sentinel.view(1, db_N, 1).expand(B, db_N, D)
h = torch.where(mask, h_sent, h_event)               # [B, db_N, D]
```

Listing 10.1: DBGNN feature initialisation.

10.4 DBGNNLayer: Message Passing on the De Bruijn Graph

Each DBGNN layer implements a SAGEConv-style update directly on $[B; |V_B|; D]$ tensors, without PyTorch Geometric overhead:

$$h_i^{(\ell+1)} = \mathrm{ReLU}\left(W_{self}^{(\ell)} h_i^{(\ell)} + W_{agg}^{(\ell)} \frac{1}{\deg(i)} \sum_{j:(j,i)\in E_B} h_j^{(\ell)} \right); \tag{10.3}$$

where $\deg(i) = |\{j : (j,i) \in E_B\}|$ is the in-degree of De Bruijn node i.

10.4.1 Scatter-Add Implementation

The mean aggregation is implemented via `scatter_add_` on a preallocated buffer, avoiding any Python loops over edges:

```
def forward(self, h, edge_index):  # h: [B, db_N, D]
    B, db_N, D = h.shape
    src, dst = edge_index[0], edge_index[1]

    h_src = h[:, src, :]                          # [B, E, D]
    dst_idx = dst.view(1, -1, 1).expand(B, -1, D)
    agg_sum = h.new_zeros(B, db_N, D)
    agg_sum.scatter_add_(1, dst_idx, h_src)

    deg = h.new_zeros(db_N)
    deg.scatter_add_(0, dst, torch.ones(E, device=h.device))
    deg = deg.clamp(min=1).view(1, db_N, 1)

    agg_mean = agg_sum / deg
    return F.relu(self.W_self(h) + self.W_agg(agg_mean))
```

Listing 10.2: DBGNNLayer forward (from gnn/dbgnn.py).

10.5 Readout

After L layers of message passing, the embedding of the end-time sentinel, $h(t_{max})$ is used as the representation of original node n:

$$h_n^{node} = h^{(L)}_{sentinel_end_indices[n]} : \tag{10.4}$$

This is a direct index lookup, not a pooling operation. The end-time sentinel receives messages from all event nodes that terminate at node n by time t_{max}, aggregating the full causal walk evidence for that node.

$$score(n) = W_{out} h_n^{node} \; 2 \; R; \tag{10.5}$$

then susceptible masking and log-softmax.

10.6 Con guration

```yaml
model: dbgnn

train:
  n_mc: 500
  reps: 3
  test_size: 0.20
  batch_size: 64
  epochs: 500
  patience: 20
  lr: 0.001
  weight_decay: 0.0005
  seed: 42

dbgnn:
  hidden_channels: 64
  num_conv_layers: 3
  conv_type: sage
  dropout_rate: 0.2
```

Listing 10.3: DBGNN con g (exp/france_office/dbgnn.yml).

10.7 Complexity Discussion

The De Bruijn graph has $|V_B| = O(|E^t|)$ nodes (one per contact event plus sentinels). For the france_office network with $|E^t| \quad 9\,600$ unique temporal edges (counting both directions), this gives approximately $19\,200$ event nodes plus $N_s = 232$ sentinel nodes. The De Bruijn edge count is bounded by $|E_B| \quad |V_B|^2$ in the worst case but is typically $O(|V_B| \; d)$ where d is the mean in-degree in B. The per-batch memory footprint is $O(B \quad |V_B| \quad D)$, which can be large for high-resolution temporal networks.

Chapter 11

DAG-GNN: Temporal Event Graph GNN

11.1 Paper References

1. Rey, Shi, Buciluä, and Smola, Directed Acyclic Graph Convolutional Networks, arXiv:2506.12218 (2025).
2. Saramäki and Holme, Weighted Temporal Event Graphs, arXiv:1912.03904 (2019).

The implementation in gnn/dag_gnn.py adapts Rey et al.'s DAG convolution to the temporal event graph representation of Saramäki et al.

11.2 Motivation: Why a DAG?

The temporal event graph (TEG) is naturally a DAG because causal edges point strictly forward in time. This has important consequences for message passing:

1. No cycles: Information cannot loop back through earlier events. This means we can process the graph in a single topological sweep.
2. Backward propagation: By reversing the DAG edges, information ows from late events (near t_{end}) toward early events (near $t = 0$, where the source is). This aligns the information ow with the source detection task.
3. Exact inference: On a DAG, belief propagation is exact (no loops to iterate over). While our model is approximate (learned weights, xed layers), the DAG structure removes one source of approximation error.

11.3 Feature Initialisation

Each event node e_{uvt} (representing the directed contact $u \to v$ at time t) is initialised by concatenating the SIR states of the source and destination nodes:

$$h_e^{(0)} = \mathrm{ReLU}\left(W_{proj} \, [x_u; x_v]^{>} \right) ; \quad W_{proj} \; 2 \; R^{D \; 6} : \tag{11.1}$$

```
x_src = x[:, ev_src, :]                # [B, n_events, 3]
x_dst = x[:, ev_dst, :]                # [B, n_events, 3]
h = F.relu(self.proj_event(
    torch.cat([x_src, x_dst], dim=-1)))  # [B, n_events, D]
```

Listing 11.1: DAG-GNN event feature initialisation.

11.4 DAG Reversal and Backward Propagation

The forward causal edges are reversed before message passing:

$$\text{rev_edge_index} = \text{ip}(\text{dag_edge_index}; \text{dim} = 0); \tag{11.2}$$

so an edge $e_1 \to e_2$ in the forward DAG (meaning $t(e_1) < t(e_2)$) becomes $e_2 \to e_1$ in the reversed DAG. In the reversed DAG, information flows from late events to early events.

11.5 DAGConvLayer

$$h_e^{(\ell+1)} = \text{ReLU}\left(W_{\text{self}}^{(\ell)} h_e^{(\ell)} + W_{\text{agg}}^{(\ell)} \frac{1}{\deg_{\text{rev}}(e)} \sum_{e' : e' \to e \text{ in rev}} h_{e'}^{(\ell)} \right). \tag{11.3}$$

In the reversed DAG, the in-neighbours of event e are all events e' such that $e \to e'$ in the forward DAG (i.e., e' happens after e and is causally enabled by e). Thus, the aggregation at e collects evidence from all events that e could have caused.

The implementation is structurally identical to DBGNNLayer, operating on the reversed edge index.

11.6 Readout: Events to Original Nodes

After L DAGConv layers, event embeddings are aggregated back to original nodes. For node v, we aggregate all events where node v was the arriving node (i.e., v received the contact):

$$h_v^{\text{node}} = \frac{1}{|\{e : \text{event_to_node}[e] = v\}|} \sum_{e : \text{event_to_node}[e] = v} h_e^{(L)}. \tag{11.4}$$

The rationale for aggregating over arriving events (not departing) is that the source node s is the origin of transmission chains — all events in which information from s eventually propagates will be on paths leading to the infected nodes. Aggregating over arriving events at each node captures the accumulated causal evidence pointing toward that node as a potential source.

```
node_emb = torch.zeros(B, N, D, device=x.device)
dst_idx  = ev_dst.view(1, -1, 1).expand(B, n_events, D)
node_emb.scatter_add_(1, dst_idx, h)        # sum over arriving
   events

```

```
# Mean normalisation
count = torch.zeros(N, device=x.device)
count.scatter_add_(0, ev_dst, torch.ones(n_events, device=x.device)
    )
count = count.clamp(min=1).view(1, N, 1)
node_emb = node_emb / count
```

Listing 11.2: DAG-GNN readout: events to nodes.

11.7 Handling Isolated Nodes

Nodes that have no arriving events (i.e., no temporal contacts pointed at them) receive a zero embedding from the scatter-add. The `clamp(min=1)` in the mean normalisation prevents division by zero. Their score after the final linear layer will be zero before the susceptible mask is applied; if they are susceptible they will be masked to $-\infty$, otherwise they will receive a finite but relatively low score.

11.8 Configuration

```yaml
model: dag_gnn

train:
  n_mc: 500
  reps: 3
  test_size: 0.20
  batch_size: 64
  epochs: 500
  patience: 20
  lr: 0.001
  weight_decay: 0.0005
  seed: 42

dag_gnn:
  hidden_channels: 64
  num_conv_layers: 3
  dropout_rate: 0.2
  agg: mean
  delta_t: null
```

Listing 11.3: DAG-GNN config (exp/france_office/dag_gnn.yml).

11.9 Effect of the Δt Parameter

When `delta_t` is set to a finite integer Δt, only causal edges (e_1, e_2) with $t(e_2) - t(e_1) \leq \Delta t$ are included in the DAG. This limits the temporal reach of causal connections and can significantly reduce the edge count for dense contact sequences. Setting $\Delta t = 1$ restricts to immediately consecutive events, which is the most conservative interpretation of causality in a discrete-time model.

Chapter 12

Training Pipeline

12.1 Data Structures

12.1.1 TSIRData

The TSIRDatadataclass (de ned insetup/data_loader.py) holds all simulation outputs for a single network and set of SIR parameters:

```
@dataclass
class TSIRData:
    config: dict              # original YAML config dict
    n_nodes: int               # N
    n_runs: int               # ground-truth runs per source
    mc_runs: int              # MC training runs per source
    mc_S:     np.ndarray       # [N, mc_runs, N] int8 -- S state
    mc_I:     np.ndarray       # [N, mc_runs, N] int8 -- I state
    mc_R:     np.ndarray       # [N, mc_runs, N] int8 -- R state
    truth_S: np.ndarray       # [N, n_runs, N] int8 -- S state
    truth_I: np.ndarray       # [N, n_runs, N] int8 -- I state
    truth_R: np.ndarray       # [N, n_runs, N] int8 -- R state
    maximal_outbreak: np.ndarray  # [N, N]
    possible: np.ndarray      # [N, n_runs, N]
    lik_possible: np.ndarray       # [N, n_runs, N]
```

Listing 12.1: TSIRData dataclass elds.

The index convention for all three-dimensional arrays is [source, run, node] : entry mc_S[s, r, v] equals 1 if node v is in state S in run r of the MC simulation started from source s.

12.1.2 SIRDataset: Constructing Training Samples

The MC simulations are unrolled into a at dataset of (observation, label) pairs:

```
# mc_S: [N, mc_runs, N] -> reshape to [N*mc_runs, N]
n_total = n_nodes * mc_runs
S = mc_S.reshape(n_total, n_nodes)
I = mc_I.reshape(n_total, n_nodes)
```

```
R = mc_R.reshape(n_total, n_nodes)
X = np.stack([S, I, R], axis=-1)    # [N*mc_runs, N, 3]

# Source label: repeat each source index mc_runs times
y = np.repeat(np.arange(n_nodes), mc_runs)  # [N*mc_runs]
# y[s * mc_runs + r] = s (source index)
```

Listing 12.2: Flattening MC simulations into training samples.

The dataset has $N \quad n_{mc}$ samples. Each sample is a tuple (X_i, y_i) where $X_i \; 2 \; f \; 0; 1g^{N \; 3}$ is the SIR state snapshot and $y_i 2 \; f0; : : : ; N \quad 1g$ is the true source index.

12.2 Train/Validation Split

The dataset is split with a strati ed train/test split (scikit-learn train_test_split) with test_size=0.20 or 0.30 depending on the model con g. Strati cation is by source node index y, ensuring that each source is represented in both train and validation sets.

Index pairs $(s; r)$ are the unit of splitting, not individual samples, preserving the dependency structure within a run.

12.3 Training Loop

Algorithm 1: Training Loop with Early Stopping

```
Input: Dataset (X; y), model f , epochs E, patience P, batch size B
Output: Best model parameters
Split (X_train; y_train); (X_val; y_val) from (X; y);
      , L_val  +1, counter  0;
for epoch = 1; : : : ; E do
    Shu e X_train;
    for each mini-batch f(X_i; y_i)g_{i2B} do
        ^_i = f  (X_i) ;                                   // forward pass
        L =  1/|B| P_i ^_i[y_i] ;                          // NLL loss
              r        L ;                                 // Adam step
    end
    L_val = 1/|X_val| P_i ^_i[y_i];
    if L_val < L_val then
          , L_val  L_val, counter  0;
    end
    else
        counter  counter +1;
        if counter  P then
            Restore  ;
            break;
        end
    end
end
return  ;
```

The Adam optimiser is used with weight decay (L2 regularisation):

$$_{t+1} = _t \quad \hat{m}_t=(\sqrt{\hat{v}_t} +) \quad _t; \tag{12.1}$$

with = 0:001, = 0:0005 (or 0:0001 for BacktrackingNetwork).

12.4 Inference on Ground-Truth Simulations

After training, the model is evaluated on the ground-truth simulations. The $N \; n_{runs}$ snapshots fromtruth_S/I/R are stacked into a tensor $X_{truth} \; 2 \; f \; 0; 1g^{N \; n_{runs} \; N \; 3}$ and passed through the model in batches of 64 to produce log-probabilities.

The log-probabilities are exponentiated to obtain probabilities and then corrected by lik_possible (see Equation 4.5).

12.5 Repetitions

Each experiment is runreps: 3 times with di erent random seeds for weight initialisation and data shu ing. Evaluation metrics are reported as mean and standard deviation across

repetitions.

Chapter 13

Evaluation Metrics

All metrics are computed in eval/scores.py . Let $p^{(r)} \in [0,1]^N$ be the predicted probability vector for run r, and let $s^{(r)}$ be the true source for run r. Let $\text{rank}^{(r)}$ be the rank (1-indexed) of the true source $s^{(r)}$ in the descending ordering of $p^{(r)}$:

$$\text{rank}^{(r)} = 1 + |\{v : p_v^{(r)} > p_{s^{(r)}}^{(r)}\}|. \tag{13.1}$$

Let sel be the set of valid runs (those with at least min_outbreak infected nodes, default 2).

13.1 Top-k Accuracy

$$\text{Top-}k = \frac{1}{|\text{sel}|} \sum_{r \in \text{sel}} \mathbb{1}[\text{rank}^{(r)} \le k]. \tag{13.2}$$

Reported for $k \in \{1, 3, 5\}$. Top-1 is the fraction of runs where the model identifies the correct source as the single most probable candidate.

13.2 Inverse Rank Score

$$\text{InvRank}(\epsilon) = \frac{1}{|\text{sel}|} \sum_{r \in \text{sel}} \frac{1+\epsilon}{\text{rank}^{(r)} + \epsilon}, \tag{13.3}$$

where $\epsilon \ge 0$ is an offset parameter (inverse_rank_offset). With $\epsilon = 0$: perfect score (rank 1) gives 1.0, rank 2 gives 0.5, rank N gives $1/N$.

13.3 Normalised Brier Score

$$\text{BS}_{\text{norm}} = \frac{1}{|\text{sel}|} \sum_{r \in \text{sel}} \frac{1}{2/N} \frac{1}{N} \sum_{v} (p_v^{(r)} - \mathbb{1}[v = s^{(r)}])^2. \tag{13.4}$$

The normalisation by $2/N$ makes the score 1.0 for a uniform distribution and 0 for a perfect prediction. Smaller is better.

13.4 Credible Set Coverage

The p-credible set for run r is the smallest set C_r of nodes such that $\sum_{v \in C_r} p_v^{(r)} \geq p$. The credible set score is:

$$CS(p) = \frac{1}{|\text{sel}|} \sum_{r \in \text{sel}} \mathbb{1}[s^{(r)} \in C_r]; \tag{13.5}$$

measuring the fraction of runs where the true source is contained in the p-credible set. A well-calibrated model with $p = 0.90$ should achieve $CS(0.90) \approx 0.90$.

13.5 Normalised Entropy

$$H_{\text{norm}} = \frac{1}{|\text{sel}|} \sum_{r \in \text{sel}} \frac{\sum_v p_v^{(r)} \log p_v^{(r)}}{\log N}. \tag{13.6}$$

This measures the concentration of the predicted distribution. Uniform gives $H_{\text{norm}} = 1.0$; a point mass gives $H_{\text{norm}} = 0$.

13.6 Practical Reporting

In the wandb logs, metrics are reported under keys such as:

- eval/top_1, eval/top_3, eval/top_5
- eval/rank_score_off0 (inverse rank with $\epsilon = 0$)
- eval/brier_score, eval/entropy

Chapter 14

Baselines

Non-ML baselines are evaluated in `main_eval.py` on the same `TSIRData` artefact as the GNN models, ensuring a fair comparison.

14.1 Uniform

$$p_v^{unif} = \frac{1}{N} \quad \forall v \in V: \tag{14.1}$$

This is the trivial baseline (random guessing without information). Expected Top-1 accuracy is $1/N$.

14.2 Degree Centrality

$$p_v^{deg} = \frac{\deg_{inf}(v)}{\sum_u \deg_{inf}(u)}; \tag{14.2}$$

where $\deg_{inf}(v)$ is the degree of node v in the subgraph induced by infected nodes $I(R)$. The rationale is that epidemic sources tend to be highly connected; high-degree nodes in the infected subgraph are more likely to have initiated the spread.

14.3 Closeness Centrality

$$p_v^{close} \propto \frac{N_{inf} - 1}{\sum_{u \in I(R)} d(v, u)}; \tag{14.3}$$

where $d(v, u)$ is the shortest-path distance on the infected subgraph. Sources are expected to be central (short paths to all infected nodes).